

Advanced Programming Project Report - Phase 1

In the Name of God

Instructor: Morteza Yousef Sanati | **Student:** Mohammad Amin Torkaman (40412358012)

Table of Contents

• Introduction	2	• Board class	3
• Classes	2	• Card class	3
• Heroes class	2	• CardResolver class	4
• Dcacula class	2	• Player class	4
• SherlockHolmes class	2	• Ftxui_Front class	4
• Sisters class	2	• Controller class	5
• Dr_Watsone class	3	• References	6
• Space class	3		

Introduction

In this phase of the project, the back-end structure and core game logic have been implemented. As the central core of the system, this part is responsible for managing the game state, rules, cards, players, and logical interactions between various components, providing a stable and well-engineered foundation for the display layers.

Classes

Heroes class

This class serves as the base and core structure for defining and managing the attributes of various heroes and characters in the game, holding their key specifications.

Key Methods:

- **DrawnCard():** Simulates drawing a card from the deck. It first checks if the deck is empty; if so, it returns 0. Otherwise, it takes the top card, adds it to the player's hand, removes it from the deck, and returns 1.
- **Damage(int amount):** Applies damage to the hero. If the health drops to or below zero, it sets the hero's alive status to false, removes the hero from their current board space, and clears the space pointer; otherwise, it subtracts the damage amount from the health.
- **discard_hand():** Randomly selects a card from the player's hand, retrieves its boost value, moves the card to the discard pile, and removes it from the hand.

- **Discard_Card(Card *card):** Searches for a specific card pointer in the hand. If found, it moves the card to the discard pile and erases it from the hand; otherwise, it throws a runtime error.

Dcacula class

The class associated with the Dracula character, implementing the specific logic, capabilities, and behaviors of this unique or villainous character during the game.

SherlockHolmes class

This class is responsible for implementing the features, skills, and behaviors of Sherlock Holmes as one of the main and strategic characters in the game.

Key Methods & Initialization:

- **SherlockHolmes Constructor:** Initializes Sherlock Holmes with core stats (name, melee attack type, 16 health, 2 movement) and populates its deck with character-specific and general cards (such as Administer Aid, Counterpunch, Feint, etc.). It then shuffles the deck randomly and draws the initial hand of 5 cards.

Sisters class

The class related to the sisters or related secondary characters, managing group interactions and specific rules regarding their presence in the gameplay.

Dr_Watsone class

This class defines and controls the capabilities and logic related to Dr. Watson, guiding his supportive or instrumental role in the game.

Space class

This class models and manages the various spaces, tiles, or zones of the game board where pieces and cards are placed.

Key Methods:

- **Space Constructor:** Initializes a specific tile on the board by setting its unique identifier number, its associated zones (vector of spaces), and its neighboring tiles.
- **get_zone():** Returns a vector containing pointers to all space objects that share the same zone as this tile, which is essential for tracking ranged attacks and area effects.
- **get_neighbor():** Returns a vector of adjacent neighbor spaces connected to this tile, used primarily for movement and positioning logic.
- **get_hero() & set_hero(Heroes *hero):** Retrieves the pointer of the hero currently occupying this space or places/removes a hero on this tile during movement actions.
- **get_number():** Returns the unique integer identifier of the space.

Board class

Represents the main game board and takes responsibility for managing tile layouts, player positions, and the overall status of the board structure.

Key Methods & Implementation:

- **Board Constructor:** Initializes the game board by resizing the spaces vector to 32 tiles and defining the precise connectivity graph, including zone associations and immediate neighbors for each individual space.
- **is_Adjacent(Space *s1, Space *s2):** Checks whether two given space pointers are directly adjacent to each other by iterating through the neighbor list of the first space, returning true if a connection exists.
- **get_spaces():** Returns a reference to the entire collection of spaces comprising the board structure.
- **SwapHeroes(Heroes *hero1, Heroes *hero2):** Manages tactical repositioning by safely swapping the positions and associated board space pointers of two heroes, throwing runtime errors if either hero is invalid or unplaced.

Card class

The base class for defining game cards, holding attributes such as card type, effects, and information related to each card.

CardResolver class

This class is responsible for interpreting, processing, and executing card effects after they are played by players, handling the card rules logic.

Key Methods & Interpretations:

- **execute():** This voluminous and key method uses a large switch-case structure on the card type (CardType) to examine what card has been played. Then, depending on which player or hero the card belongs to (such as Dracula, Sherlock Holmes, Sisters, or Dr. Watson), it applies the specific effects of that card; effects such as modifying attack and defense values (Attack_Value and Defence_Value), moving heroes on the board, applying or reducing damage and health, drawing a card from the hand (DrawnCard), or disabling opponent capabilities.
- **TryDisableCard():** This auxiliary method is designed for managing feint and neutralization cards (such as Feint). It first checks whether a valid card has been passed or not. Then it checks if the card owner is Sherlock Holmes or Dr. Watson and Sherlock is alive, it protects the card from being disabled; otherwise, by calling set_ApplyEffects(false), it neutralizes and invalidates the opponent card effects.

Player class

Manages and maintains information regarding players, scores, cards in hand, and each user's status throughout the game.

Ftxui_Front class

This class manages the interaction with the FTXUI-based presentation layer, rendering a text-based and engaging user interface in the terminal environment.

Key Methods & Interpretations:

- **Menu_()**: This method creates an interactive full-screen menu for the main game screen (including game, instructions, and exit options). It uses the FTXUI library to process keyboard events (such as the Enter key) and returns the appropriate e_Menu enumeration value based on the user's choice.
- **Players_Info_List(Player *p1, Player *p2)**: Implements a two-column form user interface for entering player information (name and age) along with a confirmation button. Upon user confirmation, it directly sets and saves the entered information onto the player objects.
- **Det_characters(Player *p1, Player *p2)**: Compares the players' ages so that the younger player gets priority in choosing a hero, then displays a menu for selecting a character (such as Dracula or Sherlock Holmes) and returns the final selection.
- **catch_place(Player *p1, Player *p2, Board *board)**: Receives the initial placement location of the heroes on the game board through a graphical menu and, considering the players' ages, assigns board positions (tiles number 3 and 23) to the heroes.
- **Graph_Box(vector &spaces)**: Renders the complete map and board graph inside a Canvas. This method visually and dynamically renders the connecting lines between spaces and the placement status of heroes on each tile (along with colors and brief info) in the terminal.
- **choose_comrad_place(Player *p1, Player *p2, Board *board)**: Manages the manual positioning of sidekicks and assistants for each player (the three sisters for Dracula and Dr. Watson for Sherlock) within permitted zones on the board and places them in the chosen tile after confirmation.
- **Attakcer_Heroes_Menu(Player *player, Board *board, Heroes *&hero)**: Extracts the list of living combatants belonging to the player (including the main hero and sidekicks) and provides a menu to select the current attacker during the turn.
- **Defender_Heroes_Menu(Player *player, Board *board, Heroes *&defender, Heroes *&Attacker)**: Filters eligible opposing defenders based on the attack type (melee or ranged) and by checking paths and tile adjacency, then renders the defender selection menu for the user.
- **Attacker_selected_card(Heroes *Attacker, Heroes *Defender, Player *p1, Player *p2, Board *board, Card *&Attacker_Card)**: Filters cards in the attacker's hand that are compatible with the attack type, displays the attack card selection menu, and finalizes the chosen card.
- **Defender_selected_card(Heroes *Attacker, Heroes *Defender, Player *p1, Player *p2, Board *board, Card *&Defender_Card)**: Extracts suitable defense cards from the defender's hand (or associated sidekicks), provides the option for a "no defense" choice, and applies the selected defense card.
- **MoveHero(Heroes *hero, Board *board, int max_distance)**: Uses graph search algorithms (BFS) to find all reachable tiles up to a specified distance from the hero's current location and presents a destination selection menu to the user.
- **SelectHero(Board *board)**: Scans and gathers all living combatants present on the game board and provides an interactive menu to select one of them for various operations.

- **PlaceHeroIntroduction = True**
- **PlaceHeroAdjacent(Heroes *hero, Heroes *target, Board *board):** Finds permitted empty adjacent tiles around a specific target and allows the user to place the desired hero in those surrounding positions.
- **DiscardCards(Heroes *dracula):** Allows the player to interactively select and discard specific codes or cards from a hero's hand (such as Dracula) to complete the card discarding process.
- **Event_Selected_Card(...):** Filters event-type cards from the player's hand, displays the event card selection interface, and prepares the card to execute special effects.

Controller class

The central controller of the game that directs program execution flow, receives inputs, updates states, and coordinates between various back-end and front-end components.

Key Methods & Interpretations:

- **set_players_character(int choice):** Assigns character roles (Dracula, Sherlock Holmes, and their companions) to the players based on the user's choice and the players' ages, determining who plays which faction.
- **get_younger_player() & get_older_player():** Compares the ages of both players (p1 and p2) to identify and return a pointer to either the younger player (user priority) or older player.
- **Initial_turn():** Sets the initial turn of the game by designating the younger player as the active turn player and the other as the opponent.
- **chane_turn, run() ...:** Alternates the active turn and runs the central game loop and execution engine.

References

- <https://cppreference.com/>
- C++ How to program Book
- <https://github.com/ArthurSonzogni/FTXUI.git>
- <https://github.com/>
- <https://claude.ai/>